Learning to Rank on the Cranfield Collection

Boolean weighted zone scoring with grid search, and real-valued weights from a linear SVM

Group *info fetch*

Information Retrieval (CS60092), Autumn 2026-27

1 Group details

Group name	Member	Roll number
info fetch	Sumit Mukherjee	23EC3AI17
	Abyan Hussain	23EC3AI19
	Kinnar Halder	23IE35001
	Avik Ghosh	23EC10099

Source code and results: https://github.com/sumitmukherjee7/cranfield-learning-to-rank

Summary. Both methods were implemented from scratch in Python (NumPy; scikit-learn for the SVM; NLTK only for the Porter stemmer). The 225 Cranfield queries were split into 157 training and 68 test queries. We first implemented both methods as specified, and then improved each one. All improvements and parameter choices were made by cross-validation on the training queries; the test queries were used only for the final numbers.

- **Method A** (Boolean weighted zone scoring, grid search): test MAP 0.2694, F1@10 0.2271. Weighting each Boolean term match by the term's IDF (**Method A+**) raises this to MAP 0.3048, F1@10 0.2588.
- **Method B** (linear SVM over the 22 specified features): test MAP 0.3248, F1@10 0.2639. Adding a pseudo-relevance-feedback feature, tuned BM25 parameters and the improved Boolean score (**Method B+**) raises this to MAP 0.3685, F1@10 0.2965.

Method B is significantly better than Method A, and Method B+ significantly better than Method B (paired t-test on AP, $p = 0.0001$ and $p = 0.0018$).

2 Problem statement

The task is ranked retrieval on the Cranfield collection: for a query, order the 1400 documents so that relevant documents come first. The ranking function is learned from the human relevance judgements. Two learning-to-rank formulations are required:

- **Method A: Boolean weighted zone scoring.** Each document has four zones z: title, author, bibliography and body. A Boolean match score $s_z(q, d)$ is computed per zone, and the ranking score is

$$\text{score}(q, d) = \sum_z g_z \, s_z(q, d), \qquad g_z \geq 0, \ \sum_z g_z = 1.$$

  The weights g_z are learned by grid search against the relevance judgements.

- **Method B: real-valued weights by a linear SVM.** Each query-document pair is described by 22 features (term statistics, TF-IDF statistics, and the Boolean, vector space and BM25 model scores). A linear SVM learns a weight vector $\mathbf{w}$, and documents are ranked by $\mathbf{w}^\top \mathbf{x}$.

Both methods are trained on a subset of the queries and evaluated on held-out test queries with precision, recall and F1. MAP, R-precision and nDCG@10 are also reported. The deliverables are the programs, text files with the ranked documents for the test queries, and this report.

3 Implementation details

3.1 Data and preprocessing

Item	Value
Documents (`cran.all.1400`)	1400; fields `.T` title, `.A` author, `.B` bibliography, `.W` body
Queries (`cran.qry`)	225; on average 9.7 distinct terms after preprocessing
Relevance judgements (`cranqrel`)	1612 relevant pairs, 7.16 per query
Vocabulary (stemmed)	6576 terms
Train / test split	157 / 68 queries (random, seed 42)

Table 1: The Cranfield collection.

Three properties of the raw files need care. (i) The `.I` numbers in `cran.qry` are not contiguous (001, 002, 004, 008, ...), while `cranqrel` numbers the queries 1 to 225 in file order, so queries are renumbered by position. (ii) `cranqrel` grades relevance with Cleverdon's codes, 1 = complete answer to 4 = minimum interest. Every query also has exactly one pair with code -1; these are not positive judgements and are treated as non-relevant. For graded learning we use grade $= 5 -$ code (4 = best), and 0 for every other document. (iii) A few documents contain a repeated field marker; the parser appends the text to the current field.

Text is lower-cased and split on non-alphanumeric characters. Stop words are removed (a SMART-style list of about 150 words), and the remaining tokens are Porter-stemmed. The same pipeline is applied to queries and documents. An in-memory index stores postings (d, tf) separately for every zone and for the whole document, together with document frequencies and zone lengths. The Cranfield body repeats the title, so title terms are naturally counted twice in the whole-document stream.

3.2 Method A: Boolean weighted zone scoring

For a query with distinct terms $t_1, \ldots, t_m$, the Boolean match of zone z is computed in one of four ways. All of them are Boolean at the level of individual terms: they only use whether t_i occurs in z, never how often.

- **fraction**: $s_z = \frac{1}{m}\left|\{i : t_i \in z\}\right|$, the proportion of query terms present in the zone (coordinate matching);
- **idf** (our improvement, Method A+): $s_z = \sum_{i:\, t_i \in z} \mathrm{idf}(t_i) \big/ \sum_i \mathrm{idf}(t_i)$, the same Boolean matches but with rare, specific terms counting more than common ones;
- **any**: $s_z = 1$ if at least one query term occurs in z (Boolean OR);
- **all**: $s_z = 1$ if every query term occurs in z (Boolean AND).

The grid search enumerates every weight vector on the simplex whose components are multiples of a step δ. With $\delta = 0.05$ that is 1771 candidates. The zone feature matrix of each training query (1400×4) is multiplied by the matrix of all candidate weights at once. Documents are then sorted, and the objective is computed with vectorised cumulative sums. Two objectives are compared:

- **MAP**: maximise mean average precision over the training queries, which directly targets ranking quality;
- **MSE**: the textbook formulation (Manning et al., *Introduction to Information Retrieval*, §6.1.2). Minimise the squared error between $\mathrm{score}(q, d)$ and the binary relevance label, with relevant and non-relevant pairs weighted equally to offset the roughly 1:190 class imbalance.

Boolean scores produce many ties. Scores are rounded to 10 decimals, so that equal Boolean evidence gives exactly equal scores whatever the order of the floating-point sums. Ties are then broken by ascending document id, so every run is deterministic.

3.3 Method B: features and linear SVM

The 22 features follow the LETOR feature set and are computed on the whole-document stream. Here $\mathrm{tf}(t, d)$ is the raw term frequency, $|d|$ the stream length and $\mathrm{idf}(t) = \log(N/\mathrm{df}_t)$. Every statistic is taken over the m query terms, including those with $\mathrm{tf} = 0$.

#	Feature	Definition
1	Query term number	number of query terms with $\mathrm{tf}(t,d) > 0$
2	Query term ratio	feature 1 / m
3	Stream length	$\lvert d\rvert$ in tokens
4	IDF	$\sum_i \mathrm{idf}(t_i)$
5-9	TF: sum, min, max, mean, variance	statistics of $\mathrm{tf}(t_i,d)$
10-14	Normalised TF: sum … variance	statistics of $\mathrm{tf}(t_i,d)/\lvert d\rvert$
15-19	TF×IDF: sum … variance	statistics of $\mathrm{tf}(t_i,d)\cdot\mathrm{idf}(t_i)$
20	Boolean model score	Method A score with the learned zone weights
21	Vector space model score	cosine of `ltc` document and query vectors
22	BM25	$\sum_i \mathrm{idf}_{\mathrm{BM}}(t_i)\,\frac{\mathrm{tf}\,(k_1+1)}{\mathrm{tf}+k_1(1-b+b\,\lvert d\rvert/\mathrm{avgdl})}$, $k_1=1.2$, $b=0.75$

Table 2: The 22 query-document features of Method B. $\mathrm{idf}_{\mathrm{BM}}(t) = \log\!\left(\frac{N-\mathrm{df}_t+0.5}{\mathrm{df}_t+0.5}+1\right)$.

Strict Boolean AND almost never fires for Cranfield queries of about ten terms (Section 5.1). So the Boolean model score (feature 20) is the Boolean weighted zone score from Method A. Its zone weights are learned on the training queries only, so no test information leaks into Method B. Features are min-max normalised to $[0,1]$ within each query (LETOR *QueryLevelNorm*); global z-scoring is evaluated as an ablation.

Two SVM formulations are implemented with scikit-learn's `LinearSVC` (L2-regularised hinge loss):

- **Pointwise.** Every (q,d) pair of a training query is one example (relevant = $+1$, others = -1), with balanced class weights. All 1400 documents per query are used. Documents are ranked by the decision value $\mathbf{w}^\top\mathbf{x}+b$.
- **Pairwise (RankSVM).** For two documents of the same query with $\mathrm{grade}(d_i) > \mathrm{grade}(d_j)$, the difference $\mathbf{x}_i - \mathbf{x}_j$ is a positive example. A random half of the pairs is negated to balance the classes, and there is no intercept. Candidate documents per query are all relevant ones, the 100 highest-BM25 non-relevant documents (hard negatives), and 200 random non-relevant documents. This formulation also uses the graded judgements (a code-1 document must outrank a code-3 one).

The formulation and the regularisation constant $C \in \{0.0003, 0.001, 0.003, 0.01, 0.1, 1\}$ are chosen together by 5-fold cross-validation over the *training* queries (criterion: MAP). The chosen model is retrained on all training queries, and its weights rank all 1400 documents for each test query.

3.4 Improvements (Method A+ and Method B+)

The basic Method B only matches BM25, its strongest input feature. All its features describe how the *original* query terms occur in a document, so no weighting of them can find relevant documents that use different words (the vocabulary mismatch problem). We tried several changes. Each was kept only if it raised the cross-validated MAP on the training queries.

1. **IDF-weighted Boolean zone match** (Method A+, and feature 20 of B+), as defined above.
2. **Tuned BM25.** $k_1 \in \{0.6, 0.9, 1.2, 1.5, 2, 2.5\}$ and $b \in \{0.3, 0.5, 0.75, 0.9\}$ are chosen by training MAP. Feature 22 then uses the tuned values.
3. **Pseudo-relevance feedback feature (RM3).** The top k documents of the initial BM25 ranking are assumed relevant. Each term gets the feedback weight $P_{\mathrm{fb}}(t) \propto \sum_{d\in\mathrm{top}\,k} P(t \mid d)\,P(d \mid q)$, with $P(t \mid d) = \mathrm{tf}(t,d)/\lvert d\rvert$ and $P(d \mid q)$ a softmax of the BM25 scores. The n best terms are mixed with the original query, $w(t) = \lambda\,[t \in q]/\lvert q\rvert + (1-\lambda)\,P_{\mathrm{fb}}(t)$, and the expanded query is scored with weighted BM25. The settings $k \in \{5, 10, 15\}$, $n \in \{20, 40, 60\}$ and $\lambda \in \{0.2, 0.3, 0.5\}$ are chosen by training MAP. This adds one feature (number 23).
4. **Zone BM25 features.** BM25 computed on the title and on the body alone (features 24 and 25).

The changes are evaluated as a cumulative "ladder" in Section 5.3. Method B+ is the ladder step with the highest cross-validated MAP; its formulation and C are selected as for Method B. In preliminary cross-validation we also tried a $\log(1+x)$ transform of all features. It helped the basic feature set slightly (+0.002 MAP) but hurt once the feedback feature was added, so it is not used. The full 21-feature blocks for the title and body streams (Table 10) added nothing.

3.5 Evaluation and output files

For a ranked list and cut-off k: $P@k = \lvert\mathrm{rel} \cap \mathrm{top}_k\rvert/k$, $R@k = \lvert\mathrm{rel} \cap \mathrm{top}_k\rvert/\lvert\mathrm{rel}\rvert$, and $F1@k = 2PR/(P+R)$. These are macro-averaged over the test queries for $k \in \{5, 10, 20\}$, with curves up to $k = 100$. We also report

R-precision, MAP, nDCG@10 (gain $2^{\text{grade}} - 1$) and 11-point interpolated precision-recall curves. Differences between systems are tested with a paired t-test and a Wilcoxon signed-rank test over the 68 test queries.

The ranked lists for the test queries are written in TREC format (`qid Q0 docid rank score tag`, top 100 per query) to `results/`:

- `methodA_test_ranking.txt`, `methodB_test_ranking.txt`: the final systems, Method A+ and Method B+;
- `methodA_basic_test_ranking.txt`, `methodB_basic_test_ranking.txt`: the methods as specified.

Running the code also writes the baseline rankings and the detailed results to the same folder. The programs are in `src/`: `cranfield.py` (parsing, index, metrics), `method_a_zone.py` (Method A), `features.py` and `method_b_svm.py` (Method B), and `run_experiments.py` (all experiments).

4 Plan of experiments

All model selection uses only the 157 training queries. The 68 test queries are used once, for the final numbers.

ID	Experiment	Question
E1	Method A: Boolean match (fraction / idf / any / all) × objective (MAP / MSE)	Which zone match and which training objective give useful weights?
E2	Method A: grid step 0.25 / 0.1 / 0.05; fixed single-zone and uniform weights	How sensitive is the optimum? Does learning beat fixed weights?
E3	Method B: pointwise vs pairwise SVM and C, by 5-fold CV on training queries	Which SVM formulation and regularisation work best?
E4	Method B ablations: normalisation, extra streams, feature subsets, negative sampling	Which features and design choices matter?
E5	Improvement ladder: idf Boolean, tuned BM25, PRF feature, zone BM25	How much does each change add?
E6	Test-set comparison of all systems and baselines; P/R/F1 vs k, PR curves, significance tests	How do the methods compare with each other and with standard models?
E7	5-fold cross-validation over all 225 queries, all parameters re-learned per fold	Are the conclusions stable beyond one train/test split?

Table 3: Experiments.

5 Results

5.1 Method A (E1, E2)

Match	Objective	title	author	biblio	body	Train MAP	Test MAP	P@10	F1@10
fraction	MAP	0.40	0.10	0.20	0.30	0.2764	0.2694	0.1926	0.2271
fraction	MSE	0.00	0.00	0.00	1.00	0.2007	0.1815	0.1500	0.1747
idf	MAP	0.30	0.35	0.10	0.25	0.3151	0.3048	0.2206	0.2588
idf	MSE	0.00	0.00	0.00	1.00	0.2533	0.2340	0.1618	0.1910
any	MAP	0.15	0.05	0.05	0.75	0.0454	0.0492	0.0338	0.0371
any	MSE	0.45	0.05	0.20	0.30	0.0449	0.0480	0.0338	0.0371
all	MAP	0.00	0.00	0.00	1.00	0.0281	0.0222	0.0132	0.0129
all	MSE	0.00	0.00	0.00	1.00	0.0281	0.0222	0.0132	0.0129

Table 4: Learned zone weights g_z (grid step 0.05) and test results, for each Boolean match definition and objective.

Only the graded matches (**fraction** and **idf**) produce a usable ranking. With about 9.7 terms per query, Boolean AND is almost never satisfied, so nearly every document scores 0. Boolean OR is satisfied by most bodies, so it barely discriminates. The MSE objective puts all weight on the body (test MAP 0.1815 for fraction, 0.2340 for idf), whereas optimising MAP directly selects $g = (0.40, 0.10, 0.20, 0.30)$ for the fraction match and $g = (0.30, 0.35, 0.10, 0.25)$ for the idf match, in the order (title, author, biblio, body). Weighting term matches by IDF raises test MAP from 0.2694 to 0.3048 (+13%, $p = 0.0002$).

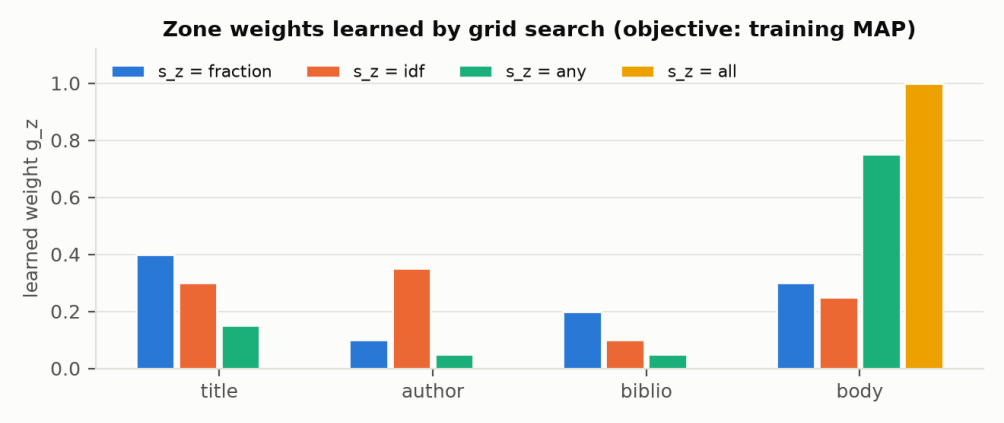


Figure 1: Zone weights learned for the four Boolean match definitions.

Zone	Relevant docs	Non-relevant docs	Ratio
title	0.8126	0.2287	3.6
author	0.0011	0.0004	2.7
biblio	0.0185	0.0039	4.8
body	0.9646	0.6262	1.5

Table 5: Fraction of documents whose zone contains at least one query term (training queries).

	fraction match		idf-weighted match	
Fixed weights	Train MAP	Test MAP	Train MAP	Test MAP
title only	0.2211	0.2391	0.2658	0.2793
author only	0.0114	0.0241	0.0114	0.0209
biblio only	0.0171	0.0138	0.0177	0.0137
body only	0.2007	0.1815	0.2533	0.2340
uniform	0.2704	0.2630	0.3120	0.3025

Table 6: Fixed reference weights: single zones and uniform weights.

Match	Step	Points	(title, author, biblio, body)	Train MAP	Test MAP
fraction	0.25	35	0.50, 0.00, 0.00, 0.50	0.2718	0.2630
fraction	0.1	286	0.40, 0.10, 0.20, 0.30	0.2764	0.2694
fraction	0.05	1771	0.40, 0.10, 0.20, 0.30	0.2764	0.2694
idf	0.25	35	0.25, 0.25, 0.25, 0.25	0.3120	0.3025
idf	0.1	286	0.50, 0.00, 0.10, 0.40	0.3130	0.3023
idf	0.05	1771	0.30, 0.35, 0.10, 0.25	0.3151	0.3048

Table 7: Effect of the grid resolution.

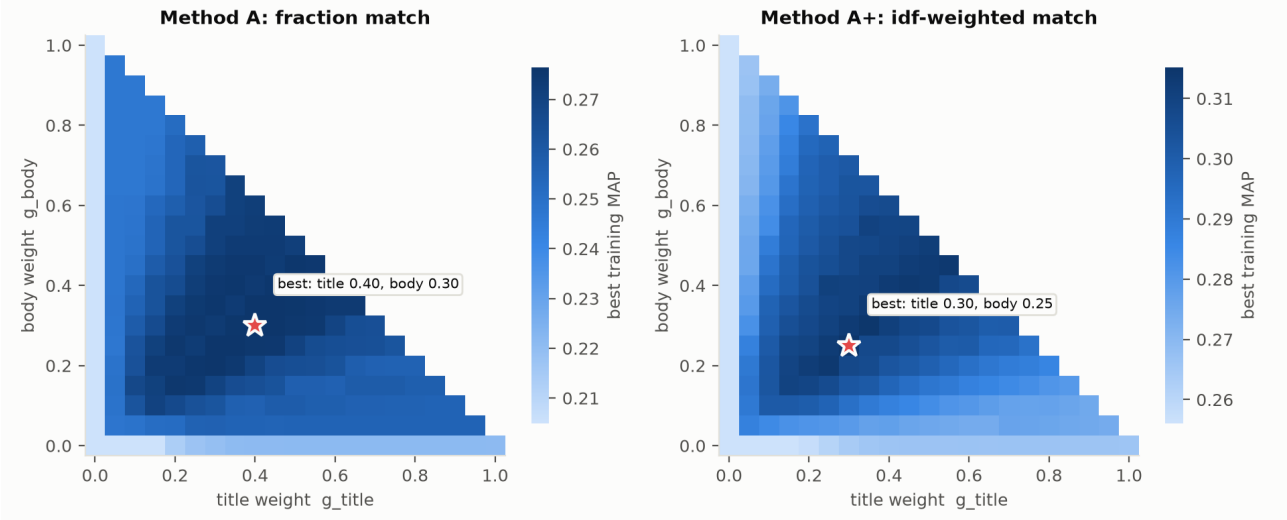


Figure 2: Training MAP over the weight simplex (each cell: best value over the remaining author/biblio split). Both optima are broad plateaus.

The objective surface is flat near the optimum. For the fraction match, the ten best grid points span training MAP 0.2759 to 0.2764, and their author and biblio weights vary widely. Those two zones contain a query term in fewer than 2% of documents, so their weights hardly change any ranking and are effectively undetermined. Title and body are what matter. A title match is 3.6 times more likely for a relevant document than for a non-relevant one, versus 1.5 times for the body. This is why the short title receives at least as much weight as the much longer body.

5.2 Method B (E3, E4)

Formulation	C=0.0003	C=0.001	C=0.003	C=0.01	C=0.1	C=1
pointwise	**0.3276**	0.3245	0.3237	0.3208	0.3192	0.3202
pairwise	**0.3250**	0.3208	0.3193	0.3190	0.3199	0.3195

Table 8: Cross-validated MAP (5 folds over the training queries) of Method B for each formulation and C. The best value for each formulation is in bold.

Formulation	C	CV MAP	Test MAP	P@10	F1@10	nDCG@10
pointwise	0.0003	0.3276	0.3248	0.2265	0.2639	0.3959
pairwise	0.0003	0.3250	0.3168	0.2353	0.2763	0.3989

Table 9: Best configuration of each formulation, retrained on all training queries and evaluated on the test queries.

The **pointwise** SVM with $C = 0.0003$ has the highest cross-validated MAP and is the Method B model. This is the strongest regularisation in the grid, but the cross-validation curve is flat: all values of C lie within about 0.01 MAP of each other.

Variant	#feat.	MAP	P@10	F1@10	nDCG@10
Method B (pointwise, query-level normalisation)	22	0.3248	0.2265	0.2639	0.3959
global z-score normalisation	22	0.3184	0.2294	0.2691	0.3956
+ title and body streams (64 features)	64	0.3252	0.2294	0.2680	0.3981
pairwise (C=0.0003), 100 hard + 200 random negatives	22	0.3168	0.2353	0.2763	0.3989
pairwise (C=0.0003), 50 hard + 50 random negatives	22	0.2966	0.2235	0.2600	0.3791
pairwise (C=0.0003), 200 hard negatives only	22	0.2786	0.2059	0.2367	0.3581
without BM25/VSM/Boolean (features 1-19)	19	0.3088	0.2206	0.2603	0.3853
without TF×IDF stats (15-19)	17	0.3190	0.2235	0.2613	0.3880
only 20-22 (Boolean, VSM, BM25)	3	0.3202	0.2235	0.2598	0.3900

Table 10: Method B ablations on the test queries (formulation and C fixed).

5.3 Improvements (E5)

On the training queries, tuning BM25 gives $k_1 = 2.5$, $b = 0.9$ (training MAP 0.3204 $\rightarrow$ 0.3281). The best feedback settings are $k = 15$ documents, $n = 60$ terms and $\lambda = 0.3$ (training MAP 0.3740). Each ladder step below adds one change to the previous step, and the SVM formulation and C are re-selected by cross-validation at every step.

Step	#feat.	SVM	C	CV MAP	Test MAP	F1@10	nDCG@10
22 features (Method B)	22	pointwise	0.0003	0.3276	0.3248	0.2639	0.3959
+ idf-weighted Boolean feature (20)	22	pointwise	0.0003	0.3296	0.3188	0.2657	0.3938
+ tuned BM25 parameters	22	pairwise	0.0003	0.3302	0.3237	0.2703	0.4010
+ PRF BM25 feature (23 features)	**23**	**pairwise**	**0.003**	**0.3748**	**0.3685**	**0.2965**	**0.4283**
+ title and body BM25 (25 features)	25	pairwise	0.003	0.3747	0.3717	0.3055	0.4368

Table 11: Improvement ladder for Method B. The bold row (highest CV MAP) is Method B+.

The feedback feature is the only step that clearly helps: it raises the cross-validated MAP from 0.3302 to 0.3748. The idf-weighted Boolean feature and the tuned BM25 parameters change the cross-validated MAP by at most 0.002, and on the test split the idf Boolean feature even lowers MAP slightly. The title and body BM25 features leave the cross-validated MAP unchanged (0.3747), so Method B+ is the 23-feature model (step "+ PRF BM25 feature (23 features)"), trained as a pairwise SVM with $C = 0.003$.

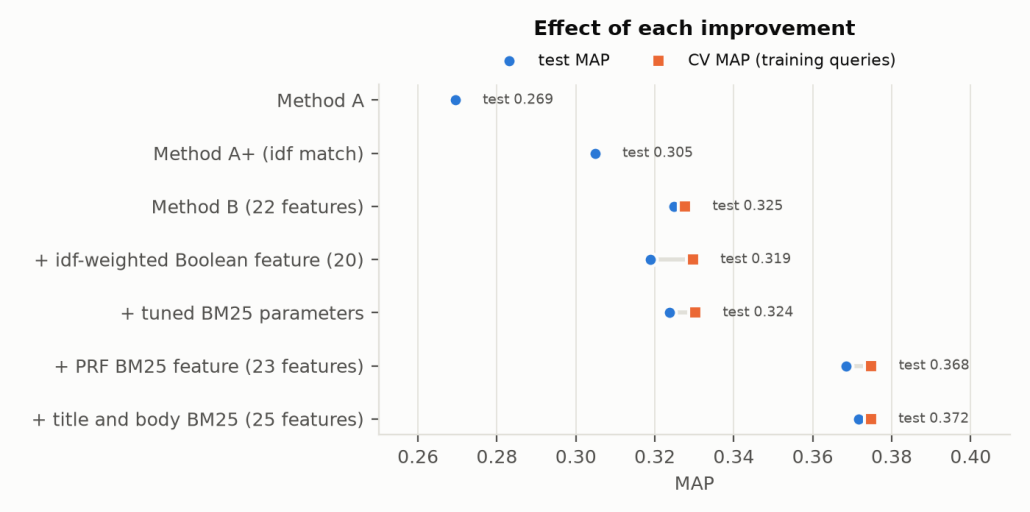


Figure 3: Cross-validated and test MAP of the basic methods and of each improvement step.

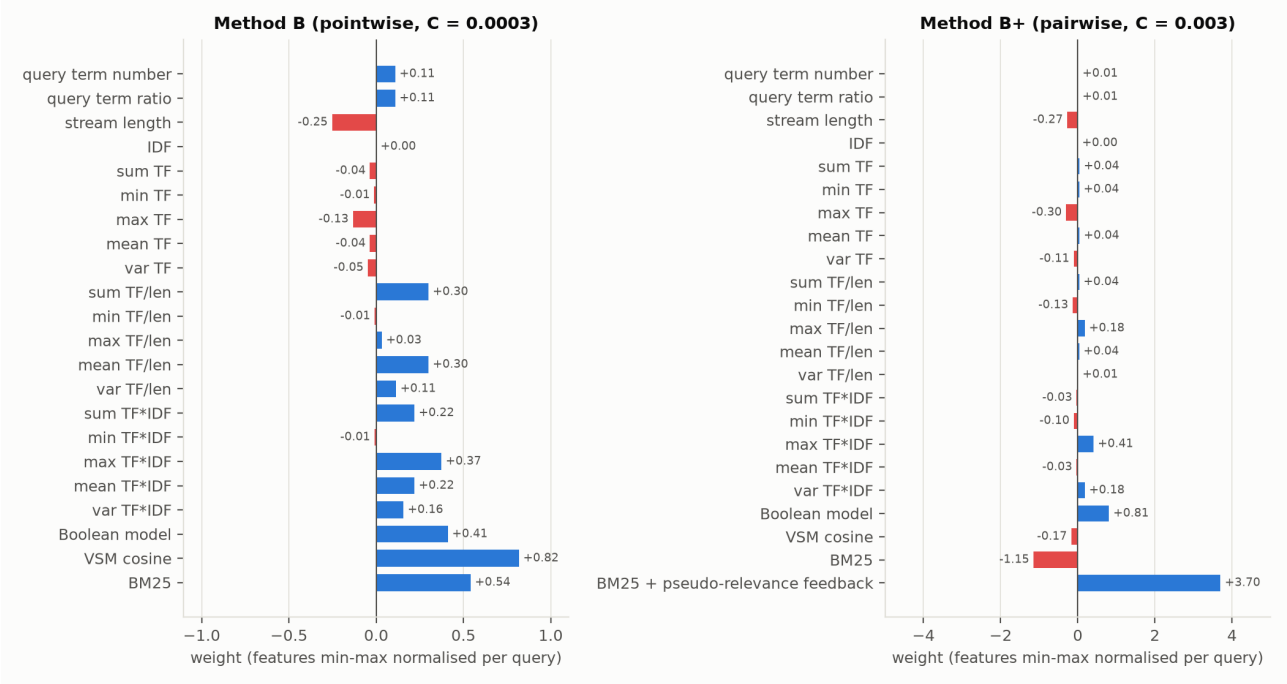


Figure 4: Learned SVM weights of Method B (left) and Method B+ (right). Blue = positive, red = negative.

5.4 Comparison on the test queries (E6)

System	P@5	R@5	F1@5	P@10	R@10	F1@10	P@20	R@20	F1@20
Boolean zone (uniform)	0.253	0.254	0.229	0.193	0.355	0.227	0.128	0.445	0.184
VSM cosine	0.306	0.284	0.264	0.218	0.391	0.254	0.150	0.502	0.214
BM25 (k1=1.2, b=0.75)	0.318	0.307	0.282	0.224	0.396	0.261	0.146	0.498	0.209
BM25 (tuned)	0.329	0.312	0.290	0.229	0.403	0.265	0.151	0.519	0.217
BM25 + PRF (tuned)	0.356	0.351	0.317	0.248	0.442	0.287	0.162	0.550	0.232
Method A (fraction match)	**0.256**	**0.258**	**0.232**	**0.193**	**0.357**	**0.227**	**0.128**	**0.443**	**0.184**
Method A+ (idf-weighted match)	**0.315**	**0.304**	**0.281**	**0.221**	**0.398**	**0.259**	**0.146**	**0.501**	**0.210**
Method B (22 features)	**0.318**	**0.304**	**0.279**	**0.227**	**0.409**	**0.264**	**0.151**	**0.513**	**0.216**
Method B+ (improved)	**0.347**	**0.342**	**0.309**	**0.256**	**0.460**	**0.296**	**0.161**	**0.557**	**0.231**

Table 12: Precision, recall and F1 at $k = 5, 10, 20$, averaged over the 68 test queries.

System	P@1	R-Prec	MAP	nDCG@10	R@50
Boolean zone (uniform)	0.3824	0.2564	0.2630	0.3390	0.5982
VSM cosine	0.3676	0.2733	0.2994	0.3724	0.6381
BM25 (k1=1.2, b=0.75)	0.3676	0.3122	0.3187	0.3853	0.6260
BM25 (tuned)	0.3971	0.3099	0.3243	0.3913	0.6544
BM25 + PRF (tuned)	0.3824	0.3404	0.3619	0.4178	0.6992
Method A (fraction match)	**0.3676**	**0.2686**	**0.2694**	**0.3395**	**0.6067**
Method A+ (idf-weighted match)	**0.3676**	**0.3059**	**0.3048**	**0.3808**	**0.6320**
Method B (22 features)	**0.4118**	**0.3022**	**0.3248**	**0.3959**	**0.6583**
Method B+ (improved)	**0.3824**	**0.3444**	**0.3685**	**0.4283**	**0.7045**

Table 13: Rank-based measures on the test queries.

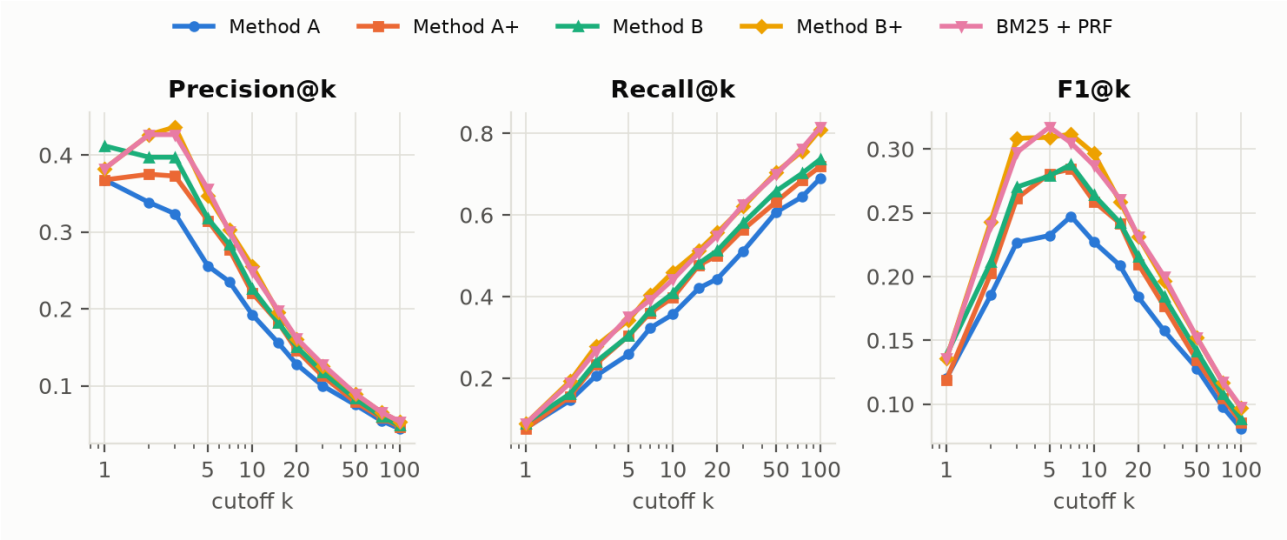


Figure 5: Precision, recall and F1 as a function of the cut-off k (log scale).

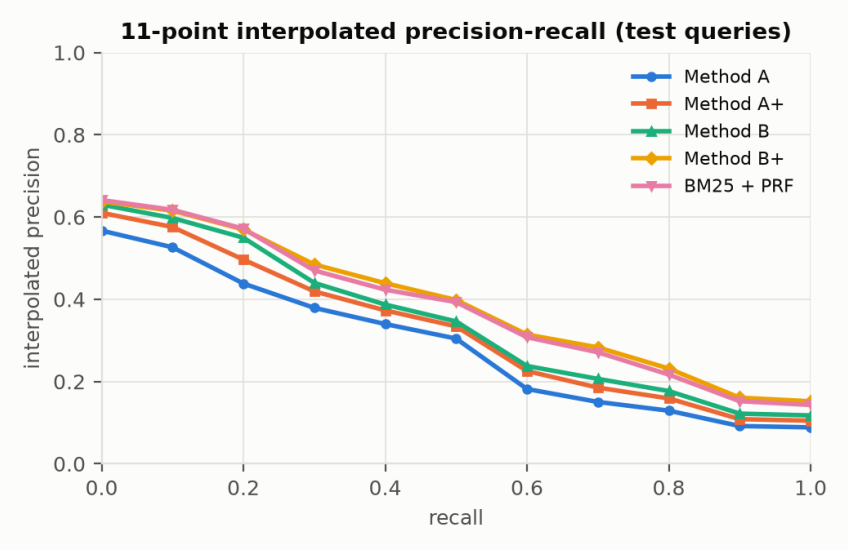


Figure 6: 11-point interpolated precision-recall curves on the test queries.

Comparison	Metric	Mean 1st	Mean 2nd	Wins/losses	t-test p	Wilcoxon p
B vs A	MAP	0.3248	0.2694	42 / 23	0.0001	0.0003
B vs A	F1@10	0.2639	0.2271	23 / 11	0.0131	0.0235
A+ vs A	MAP	0.3048	0.2694	43 / 19	0.0002	0.0001
A+ vs A	F1@10	0.2588	0.2271	21 / 6	0.0023	0.0031
B+ vs B	MAP	0.3685	0.3248	44 / 22	0.0018	0.0029
B+ vs B	F1@10	0.2965	0.2639	26 / 10	0.0250	0.0359
B+ vs A+	MAP	0.3685	0.3048	44 / 22	0.0012	0.0003
B+ vs A+	F1@10	0.2965	0.2588	27 / 12	0.0273	0.0339
B vs BM25	MAP	0.3248	0.3187	38 / 27	0.4654	0.1710
B vs BM25	F1@10	0.2639	0.2609	14 / 13	0.7631	0.6479
B+ vs BM25	MAP	0.3685	0.3187	46 / 21	0.0021	0.0022
B+ vs BM25	F1@10	0.2965	0.2609	25 / 13	0.0256	0.0232
B+ vs BM25+PRF	MAP	0.3685	0.3619	35 / 27	0.1474	0.1887
B+ vs BM25+PRF	F1@10	0.2965	0.2871	10 / 5	0.1694	0.1727
A vs Uniform zone	MAP	0.2694	0.2630	35 / 26	0.1868	0.3712
A vs Uniform zone	F1@10	0.2271	0.2266	2 / 4	0.9375	0.9165

Table 14: Paired significance tests over the test queries (per-query AP and F1@10).

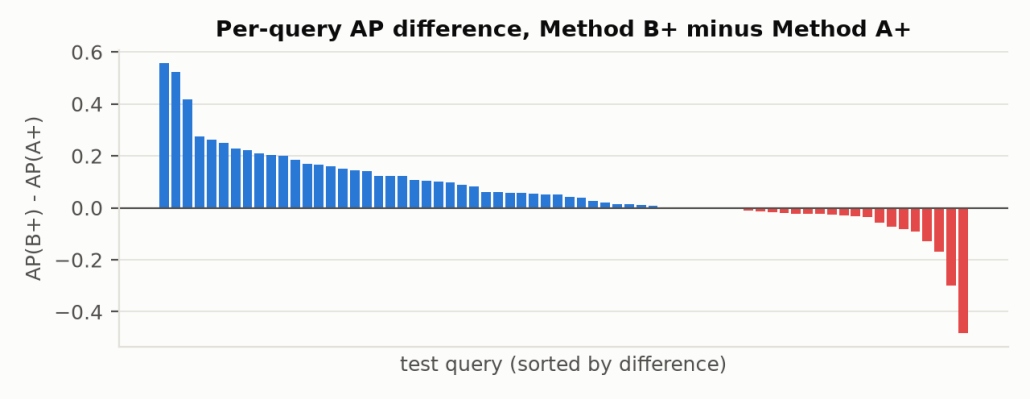


Figure 7: Per-query AP difference between Method B+ and Method A+ (B+ better on 44, worse on 22 of 68 queries).

5.5 Cross-validation over all queries (E7)

System	P@10	R@10	F1@10	MAP	nDCG@10
Method A	0.218 ± 0.011	0.372 ± 0.017	0.248 ± 0.010	0.273 ± 0.015	0.330 ± 0.006
Method A+	0.237 ± 0.013	0.402 ± 0.017	0.269 ± 0.014	0.310 ± 0.013	0.362 ± 0.015
Method B	0.245 ± 0.016	0.412 ± 0.030	0.277 ± 0.019	0.321 ± 0.028	0.370 ± 0.022
Method B+	0.282 ± 0.012	0.469 ± 0.029	0.319 ± 0.016	0.369 ± 0.028	0.419 ± 0.017
BM25	0.241 ± 0.012	0.403 ± 0.026	0.272 ± 0.015	0.320 ± 0.028	0.368 ± 0.023
BM25 + PRF (tuned)	0.281 ± 0.015	0.464 ± 0.025	0.315 ± 0.019	0.368 ± 0.028	0.413 ± 0.018

Table 15: 5-fold cross-validation over all 225 queries (mean ± standard deviation over folds). Zone weights, BM25 and PRF parameters and SVM weights are re-learned in every fold.

Over the five folds, Method B beats Method A in 5, Method A+ beats Method A in 5, and Method B+ beats Method B in 5. Method B+ beats the BM25 + PRF baseline in 3 of 5 folds.

6 Discussion of results

Method A learns little beyond "match the title and the body". Grid search behaves as expected: with the fraction match, the selected weights raise training MAP from 0.2704 (uniform weights) to 0.2764. On the test queries they score 0.2694 versus 0.2630 for uniform weights, which is not significant ($p = 0.1868$). There are two reasons the gain is small. First, only two of the four zones carry information: author and bibliography virtually never contain query terms, because Cranfield queries are topical questions. Second, a zone-level Boolean score has few distinct values, so many documents tie, and changing the weights only reorders a handful of documents. Because of these ties, even the tie-breaking rule matters. In a first version of our program, floating-point noise of about 10^{-17} decided the order of tied documents; grid search then selected different weights, and test MAP was 0.2600. The experiment also shows that the training objective matters. The squared-error objective of the textbook formulation rewards calibrated scores rather than a good ordering, and under heavy class imbalance it picks a degenerate body-only solution.

IDF weighting fixes the weakest part of Method A. Coordinate matching treats a match on "flow" the same as a match on "aeroelastic". Weighting each Boolean match by IDF keeps the method Boolean (only presence is used) but makes matches on specific technical terms count more. This increases test MAP by 13% and is the single most useful change to Method A. It also makes the learned zone weights matter less: uniform weights with the idf match already reach 0.3025.

Method B is the better ranker. Real-valued features capture how often and how specifically a query term occurs, not just whether it occurs. Method B improves on Method A by 21% in MAP ($p = 0.0001$, better on 42 and worse on 23 test queries), and it wins in 5 of the 5 cross-validation folds. The learned weights make sense. The largest weights go to the classic retrieval scores: VSM cosine (+0.82), BM25 (+0.54), max TF×IDF (+0.37) and the Boolean zone score (+0.41). Raw TF statistics and stream length get negative weights (stream length -0.25, max TF -0.13), while their length-normalised counterparts are mostly positive (sum TF/len +0.30). In effect the SVM learns document-length normalisation from the data. The IDF feature gets weight 0: it depends

only on the query, so it is constant within a query, and min-max normalisation maps it to 0. Pairs such as sum/mean TF and query term number/ratio are perfectly collinear after query-level normalisation, so the SVM gives both members of a pair the same weight.

Why the basic Method B only matches BM25. On the test queries Method B and BM25 are level (MAP 0.3248 vs 0.3187, $p = 0.4654$). The ablations explain why. Features 20-22 alone already reach 0.3202 MAP, while features 1-19 alone give 0.3088. All 22 features are functions of the same term-frequency statistics of the original query terms, so they are highly correlated and add little information beyond BM25. Extra title and body streams with the same statistics do not help either.

New evidence is what improves Method B. The pseudo-relevance-feedback feature is the first feature that brings in information the other 22 lack. It adds terms that co-occur with the query in the top-ranked documents, and so finds relevant documents that use different words. This one step gives almost all of the improvement of Method B+ over Method B (+13% MAP, +12% F1@10, $p = 0.0018$). In Method B+ the SVM gives the PRF feature by far the largest weight (+3.70). It also gives plain BM25 a *negative* weight (-1.15), so the model ranks documents partly by how much the feedback terms add on top of the original query. The PRF score on its own is, however, almost as good (BM25 + PRF: test MAP 0.3619, CV MAP 0.3676). Method B+ is level with it on the test split ($p = 0.1474$) and beats it in only 3 of the 5 cross-validation folds (CV MAP 0.3687). So most of the improvement comes from the new evidence, not from the learning. The SVM combines the PRF score with the other features without losing anything, but it gains little on top of it.

Precision, recall and F1. As k grows, precision falls and recall rises for all systems. F1 peaks at $k \approx 5$ to 10, which matches the average of 7.2 relevant documents per query. At $k = 10$ Method B+ retrieves 46% of the relevant documents at 26% precision, against 41%/23% for Method B and 36%/19% for Method A. Precision is low in absolute terms because many queries have only 2 to 5 relevant documents, which caps P@10 for those queries.

Limitations. The main comparison uses a single 157/68 split of only 225 queries, so small differences are within noise; experiment E7 was added for this reason. In E7 the SVM formulation and C were fixed to the values chosen on the main training split, while all other parameters were re-learned per fold. Several tuned values lie at the edge of their grids (C, BM25 k_1 and b, and the number of feedback documents and terms), so somewhat better settings may exist outside the ranges we searched. Pairs with code -1 were treated as non-relevant, and unjudged documents are assumed non-relevant. This is standard for Cranfield and affects all systems equally.

Conclusion. Learning zone weights by grid search is simple and interpretable, but coarse Boolean evidence limits it. IDF-weighted Boolean matches make it substantially better. A linear SVM over real-valued term statistics learns sensible weights, including length normalisation, and is significantly better than the Boolean zone model. However, it cannot go beyond BM25 with features that all describe the original query terms. Adding pseudo-relevance feedback as a feature gives Method B+ the largest gain of all the changes we tried: test MAP 0.3248 $\rightarrow$ 0.3685 and F1@10 0.2639 $\rightarrow$ 0.2965. Most of this gain comes from the feedback signal itself, which the SVM learns to rely on.